

CHAPTER – X

COLLECTIONS

JAVA.UTIL PACKAGES:

- The java.util package is one of the most important built-in packages in Java.
- The utility classes provided by util packages:
 - Data storage (Collections)
 - Working with dates
 - Working with random numbers
 - Reading input
 - Handling arrays
 - Using scanners, lists, maps, sets, etc.

ADVANTAGES:

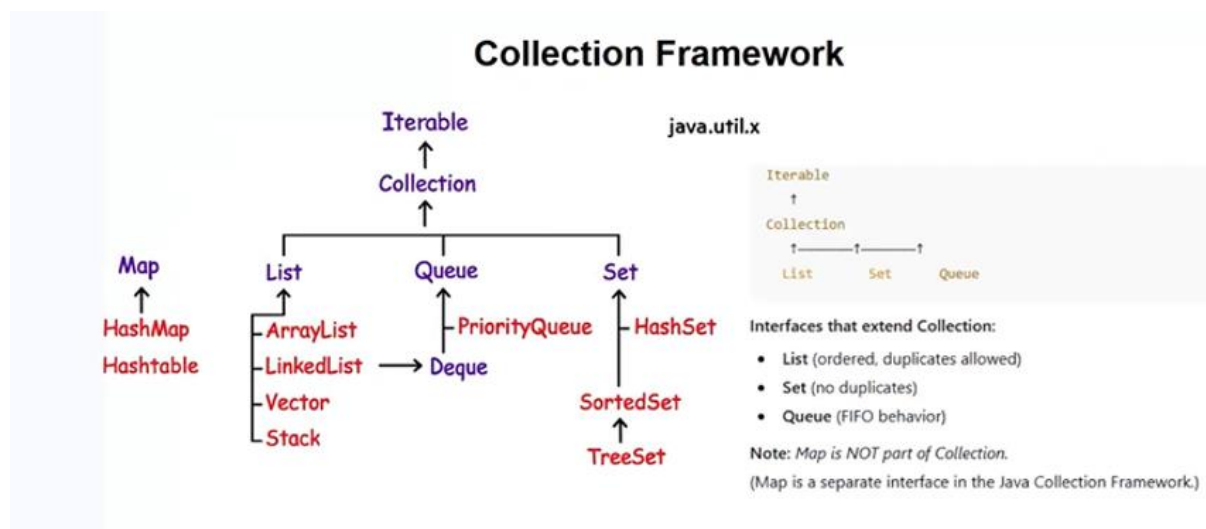
- Storing multiple objects easily (ArrayList, HashMap)
- Sorting data (Collections.sort)
- Getting user input (Scanner)
- Working with dates and times (Date, Calendar)
- Generating random numbers (Random)

Common Classes in java.util Package

Class Name	Purpose
Scanner	To take input from user
ArrayList	Growable array
HashMap	Key-value pair map
HashSet	Set of unique values
LinkedList	List with nodes
Collections	Utility methods (sort, reverse...)
Date	Work with dates
Random	Generate random values
Arrays	Utility methods for arrays

COLLECTION FRAMEWORK:

- Java Collections Framework is a set of classes and interfaces that help store, manage and process groups of objects efficiently.
- Resizable data structures
- Built-in algorithms (sorting, searching, etc.)
- Easy data processing
- Provides ready-made classes like ArrayList, HashSet, HashMap



COLLECTIONS IN JAVA:

- Collections (with s) is a utility class in the Java Collection Framework.
- It is located in java.util.Collections
- It contains static helper methods that make it easier to work with collection objects like List, Set, and Queue.

KEYS USES IN COLLECTIONS:

- **Sorting** - Collections.sort(list);
- **Reversing a list** - Collections.reverse(list);
- **Shuffling** - Collections.shuffle(list);
- **Finding maximum / minimum** - Collections.max(list);
Collections.min(list);

- **Making a collection thread-safe -**
Collections.synchronizedList(list);
- **Making a collection read-only -**
Collections.unmodifiableList(list);

Syntax of Collection (Interface)

✓ Importing Collection

```
import java.util.Collection;
```

✓ Creating a Collection (using a class like ArrayList)

```
Collection<String> names = new ArrayList<>();
```

✓ Common Methods

```
names.add("John");
```

```
names.remove("John");
```

```
names.size();
```

```
names.clear();
```

```
names.contains("John");
```

LIST INTERFACE:

- List is a subinterface of Collection in Java.
- It represents an ordered collection where:
 - Duplicates are allowed
 - Elements can be accessed by index (0-based)
 - Maintains insertion order

SYNTAX:

```
List<DataType> listName=new ArrayList<>();
```

ARRAYLIST:

- A dynamic array that grows automatically.
- Allows duplicates and maintains insertion order.
- Fast when accessing elements (index-based access).
- Slower when inserting/deleting in the middle.

Method	Description
<code>add(E e)</code>	Add element
<code>add(int index, E e)</code>	Add at index
<code>get(int index)</code>	Get element
<code>set(int index, E e)</code>	Replace element
<code>remove(int index)</code>	Remove at index
<code>remove(Object o)</code>	Remove by value
<code>size()</code>	Number of elements
<code>contains(Object o)</code>	Check if exists
<code>isEmpty()</code>	Check empty
<code>clear()</code>	Remove all elements

EXAMPLE:

```
import java.util.*;

public class ArrayListEg {

    public static void main(String [] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("Apple");

        list.add("Banana");

        list.add("Grapes");

        list.add("Mango");

        System.out.println("List: "+list);

        list.add(1,"Dragon Fruit");
```

```

        System.out.println("List After Insertion" +list);
        System.out.println("Element at index 3: "+list.get(4));
        System.out.println("Checks if the list is empty:
"+list.isEmpty());
    }
}

```

LINKEDLIST:

- A doubly linked list is a linked list where each node contains data and two pointers-one pointing to the next node and another pointing to the previous node.
- Based on doubly-linked list.
- Best for insertion and deletion.
- Slower for random access (get()).

LinkedList has all List methods + these:

Method	Description
<code>addFirst(E e)</code>	Add at beginning
<code>addLast(E e)</code>	Add at end
<code>getFirst()</code>	First element
<code>getLast()</code>	Last element
<code>removeFirst()</code>	Remove first
<code>removeLast()</code>	Remove last
<code>offer(E e)</code>	Queue insert
<code>poll()</code>	Queue remove

EXAMPLE:

```
import java.util.*;

public class LinkedListEg {
    public static void main(String [] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("Abi");
        list.add("Anu");
        list.addFirst("Banu");
        list.addLast("priya");
        System.out.println("List: "+list);
        System.out.println("First element: "+list.getFirst());
        System.out.println("Last element: "+list.getLast());
        list.removeFirst();
        list.removeLast();
        System.out.println("List After updating: "+list);
    }
}
```

VECTOR:

- Vector is a dynamic array similar to ArrayList but synchronized, meaning it is thread-safe.

KEY POINTS:

- Thread-safe (synchronized)
- Slower than ArrayList due to synchronization
- Maintains insertion order
- Allows duplicates
- Grows by doubling its size when needed

➤ Legacy class (older, used before ArrayList)

Method	Description
<code>capacity()</code>	Current capacity
<code>addElement(E obj)</code>	Add element
<code>elementAt(int index)</code>	Get element
<code>removeElement(Object o)</code>	Remove by value
<code>removeAllElements()</code>	Clear all

EXAMPLE:

```
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {
        Vector<String> fruits = new Vector<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");
        System.out.println("Vector elements: " + fruits);
        System.out.println("First element: " + fruits.get(0));
        fruits.remove("Banana");
        System.out.println("After removing Banana: " + fruits);
    }
}
```

STACK:

- Stack is a subclass of Vector that follows the LIFO (Last In, First Out) order for storing and retrieving elements.
- **KEY POINTS:**
 - Uses LIFO (Last In, First Out)
 - Supports push, pop, peek operations
 - Useful for undo, recursion, expression evaluation
 - Thread-safe because it inherits from Vector
 - Slower due to synchronization
 - Legacy class (modern alternative: Deque)

Method	Description
<code>push(E e)</code>	Add element
<code>pop()</code>	Remove & return top element
<code>peek()</code>	Return top without removing
<code>empty()</code>	Check if empty
<code>search(Object o)</code>	Position from top

SET INTERFACE:

- Set is a subinterface of Collection in Java that represents a group of unique elements.
- **KEY POINTS:**
 - Does NOT allow duplicates
 - Does NOT maintain insertion order (except LinkedHashSet)
 - At most one null element (TreeSet does NOT allow null)
- **SYNTAX:**

```
Set<Type>set=new HashSet<>();
```


Common Implementations of Set

Implementation	Features
HashSet	Fastest; no order maintained
LinkedHashSet	Maintains insertion order
TreeSet	Sorted order; no null values

Important Methods

★ Common Set Methods (Available in all Set types)

Method	Description
<code>add(E e)</code>	Adds an element (ignored if duplicate)
<code>remove(Object o)</code>	Removes element
<code>contains(Object o)</code>	Checks if element exists
<code>size()</code>	Returns number of elements
<code>clear()</code>	Removes all elements
<code>isEmpty()</code>	Checks if empty
<code>iterator()</code>	Iterates through the set

HASHSET:

- Stores elements in random order
- Fastest set implementation
- Allows null value

EXAMPLE:

```
import java.util.*;

public class HashSetEg {
    public static void main(String [] args) {
        HashSet<String> hs = new HashSet<>();
```

```

        hs.add("Dog");
        hs.add("Cat");
        hs.add("Elephant");
        hs.add("Lion");
        System.out.println("HashSet: "+hs);
        System.out.println("Size of the HashSet: "+hs.size());
        System.out.println("Checks the element is available:
        "+hs.contains("Lion"));
    }
}

```

LINKEDHASHSET:

- Keeps elements in the same order they were inserted
- Slightly slower than HashSet

EXAMPLE:

```

import java.util.*;

public class LinkedHashSetEg {
    public static void main(String [] args) {
        LinkedHashSet<Integer> lhs = new LinkedHashSet<>();
        lhs.add(50);
        lhs.add(20);
        lhs.add(10);
        lhs.add(30);
        lhs.add(40);
        lhs.add(50);
        System.out.println("LinkedHashSet: "+lhs);
    }
}

```

```
        System.out.println("checks the element:
"+lhs.contains(30));

        System.out.println("First element: "+lhs.getFirst());

    }

}
```

TREESET(SORTED SET):

- Stores elements in sorted (ascending) order
- Does NOT allow null
- Uses Red-Black Tree internally

EXAMPLE:

```
import java.util.TreeSet;

public class TreeSetDemo {

    public static void main(String[] args) {

        TreeSet<Integer> numbers = new TreeSet<>();

        numbers.add(50);
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);
        numbers.add(10);

        System.out.println(numbers);

    }

}
```